Large Scale System Design

High-Level Design (HLD)

Systems 9 - 12

Design YouTube - Design WhatsApp
Design Instagram Feed - Design Twitter Timeline

YouTube | WhatsApp | Instagram | Twitter

Table of Contents

System	Section	Topics Covered
09	Design YouTube	Upload pipeline · Transcoding · HLS streaming · Search · Recommendations
10	Design WhatsApp	WebSocket gateway · Message delivery · E2E encryption · Presence
11	Design Instagram Feed	Fan-out · Feed ranking · Stories · Snowflake IDs · Celebrity problem
12	Design Twitter Timeline	Tweet creation · Fan-out service · Trending · Count-Min Sketch · Search

Large Scale System Design (HLD) — Systems 9–12

SYS 9
#09

System 9: Design YouTube

9.1 Requirements

REQUIREMENTS

```
Functional:
  • Upload videos
  • Stream/watch videos (adaptive bitrate)
  • Search videos
  • Like, comment, subscribe
  • Recommendations

Non-Functional:
  • High availability (99.99%)
  • Low latency streaming (<200ms start)
  • Support 1B+ DAU, 500 hrs video uploaded/min
  • Global reach via CDN
  • Eventual consistency is acceptable
```

9.2 Capacity Estimation

CAPACITY

```
DAU:                1 Billion
Videos watched/day: 5 per user → 5B views/day
Avg video size:     300 MB (original)
Uploads/day:        500K videos
Storage/day:        500K × 300MB = 150 TB (raw)
                    With multiple resolutions: ~500 TB/day
Bandwidth:
  Streaming: 5B × 50MB avg = 250 PB/day ≈ 23 Gbps average
```

9.3 High-Level Architecture

ARCHITECTURE

ARCHITECTURE

```
■   Client     ■  (Web / Mobile / Smart TV)
■■■■■■■■■■■■■■■■■
      ■
      ▼
■■■■■■■■■■■■■■■■■
```

ARCHITECTURE

```
description     TEXT
category        VARCHAR(50)
tags            JSON              -- ["music","pop"]
duration_sec    INT
status          ENUM('uploading','processing','live','failed','deleted')
visibility      ENUM('public','unlisted','private')
view_count      BIGINT DEFAULT 0
like_count      BIGINT DEFAULT 0
dislike_count   BIGINT DEFAULT 0
upload_time     DATETIME
thumbnail_url   VARCHAR(500)
═══════════════════════════════════════════════════

Table: video_files
═══════════════════════════════════════════════════
video_id        VARCHAR(11) FK
resolution      VARCHAR(10)       -- "1080p", "720p", "480p", "360p"
codec           VARCHAR(20)       -- "h264", "vp9", "av1"
bitrate_kbps    INT
file_url        VARCHAR(500)      -- S3 path
file_size_bytes BIGINT
PK(video_id, resolution, codec)
═══════════════════════════════════════════════════

Table: users
═══════════════════════════════════════════════════
user_id         BIGINT PK
username        VARCHAR(100) UNIQUE
email           VARCHAR(200) UNIQUE
channel_name    VARCHAR(200)
subscriber_count BIGINT DEFAULT 0
created_at      DATETIME
═══════════════════════════════════════════════════

Table: comments
═══════════════════════════════════════════════════
comment_id      BIGINT PK
video_id        VARCHAR(11) FK
user_id         BIGINT FK
parent_id       BIGINT NULL       -- for replies
content         TEXT
like_count      INT DEFAULT 0
created_at      DATETIME
═══════════════════════════════════════════════════

Table: subscriptions
═══════════════════════════════════════════════════
subscriber_id   BIGINT
channel_id      BIGINT
subscribed_at   DATETIME
PK(subscriber_id, channel_id)
═══════════════════════════════════════════════════
"""
```

9.5 Core Flows — Python Examples

Video Upload Flow

PYTHON

```python
import uuid
```

PYTHON

```python
import hashlib
import boto3
from fastapi import FastAPI, UploadFile, File, Depends
from confluent_kafka import Producer
import json

app = FastAPI()
s3 = boto3.client('s3', region_name='us-east-1')
kafka_producer = Producer({'bootstrap.servers': 'kafka:9092'})

BUCKET_RAW = "youtube-raw-videos"
BUCKET_PROCESSED = "youtube-processed-videos"
CHUNK_SIZE = 5 * 1024 * 1024  # 5MB for multipart

# ■■■ Generate unique video ID (YouTube style: 11 chars) ■■■
def generate_video_id() -> str:
    """YouTube uses 11-character base64 IDs → 64^11 ≈ 7.3 × 10^19 possibilities."""
    raw = uuid.uuid4().bytes
    encoded = hashlib.md5(raw).hexdigest()[:11]
    return encoded

# ■■■ Resumable Upload (Chunked) ■■■
class UploadSession:
    """Manages resumable upload sessions."""

    def __init__(self, video_id: str, total_size: int):
        self.video_id = video_id
        self.total_size = total_size
        self.uploaded_bytes = 0
        self.parts = []

        # Initiate S3 multipart upload
        response = s3.create_multipart_upload(
            Bucket=BUCKET_RAW,
            Key=f"raw/{video_id}/original"
        )
        self.upload_id = response['UploadId']

    def upload_chunk(self, chunk: bytes, part_number: int) -> dict:
        """Upload a single chunk."""
        response = s3.upload_part(
            Bucket=BUCKET_RAW,
            Key=f"raw/{self.video_id}/original",
            UploadId=self.upload_id,
            PartNumber=part_number,
            Body=chunk
        )
        self.parts.append({
            'PartNumber': part_number,
            'ETag': response['ETag']
        })
        self.uploaded_bytes += len(chunk)
        return {
            'part_number': part_number,
            'uploaded': self.uploaded_bytes,
            'total': self.total_size,
            'progress': round(self.uploaded_bytes / self.total_size * 100, 1)
```

PYTHON

```python
        }

    def complete(self):
        """Complete multipart upload."""
        s3.complete_multipart_upload(
            Bucket=BUCKET_RAW,
            Key=f"raw/{self.video_id}/original",
            UploadId=self.upload_id,
            MultipartUpload={'Parts': sorted(self.parts, key=lambda x: x['PartNumber'])}
        )

# Store active sessions
upload_sessions: dict[str, UploadSession] = {}

@app.post("/api/v1/videos/upload/init")
async def init_upload(title: str, description: str, file_size: int, user_id: int):
    """
    Step 1: Client requests an upload URL.
    Returns a session with video_id for chunked upload.
    """
    video_id = generate_video_id()

    # Save metadata to DB with status='uploading'
    await save_video_metadata(video_id, user_id, title, description, status='uploading')

    session = UploadSession(video_id, file_size)
    upload_sessions[video_id] = session

    return {
        "video_id": video_id,
        "upload_url": f"/api/v1/videos/upload/{video_id}/chunk",
        "chunk_size": CHUNK_SIZE,
        "total_parts": (file_size + CHUNK_SIZE - 1) // CHUNK_SIZE
    }

@app.put("/api/v1/videos/upload/{video_id}/chunk")
async def upload_chunk(video_id: str, part_number: int, chunk: UploadFile = File(...)):
    """Step 2: Upload individual chunks (supports resume on failure)."""
    session = upload_sessions[video_id]
    data = await chunk.read()
    result = session.upload_chunk(data, part_number)
    return result

@app.post("/api/v1/videos/upload/{video_id}/complete")
async def complete_upload(video_id: str):
    """Step 3: Finalize upload → trigger processing pipeline."""
    session = upload_sessions.pop(video_id)
    session.complete()

    # Update status
    await update_video_status(video_id, 'processing')

    # Publish to Kafka for async processing
    event = {
        "event": "video.uploaded",
        "video_id": video_id,
        "s3_path": f"s3://{BUCKET_RAW}/raw/{video_id}/original",
```

PYTHON

```python
        "timestamp": "2024-01-15T10:30:00Z"
    }
    kafka_producer.produce(
        topic='video-processing',
        key=video_id.encode(),
        value=json.dumps(event).encode()
    )
    kafka_producer.flush()

    return {"video_id": video_id, "status": "processing"}
```

Video Processing Pipeline (Transcoding)

ARCHITECTURE

```python
"""
Video Processing Pipeline
═══════════════════════════
Triggered by Kafka consumer, uses FFmpeg for transcoding.

Pipeline stages:
  1. Download raw video from S3
  2. Validate & extract metadata
  3. Transcode to multiple resolutions
  4. Generate thumbnails
  5. Upload processed files to S3
  6. Update metadata DB
  7. Propagate to CDN
"""

import subprocess
import os
import json
from dataclasses import dataclass
from concurrent.futures import ThreadPoolExecutor
from confluent_kafka import Consumer

@dataclass
class TranscodeProfile:
    name: str
    resolution: str
    width: int
    height: int
    video_bitrate: str
    audio_bitrate: str

PROFILES = [
    TranscodeProfile("2160p", "3840x2160", 3840, 2160, "15000k", "192k"),
    TranscodeProfile("1080p", "1920x1080", 1920, 1080, "5000k",  "128k"),
    TranscodeProfile("720p",  "1280x720",  1280, 720,  "2500k",  "128k"),
    TranscodeProfile("480p",  "854x480",   854,  480,  "1000k",  "96k"),
    TranscodeProfile("360p",  "640x360",   640,  360,  "600k",   "96k"),
    TranscodeProfile("240p",  "426x240",   426,  240,  "400k",   "64k"),
]

class VideoProcessor:
    def __init__(self, video_id: str, input_path: str):
        self.video_id = video_id
        self.input_path = input_path
        self.output_dir = f"/tmp/processed/{video_id}"
        os.makedirs(self.output_dir, exist_ok=True)

    def extract_metadata(self) -> dict:
```

ARCHITECTURE

```
        """Use ffprobe to get video info."""
        cmd = [
            'ffprobe', '-v', 'quiet',
            '-print_format', 'json',
            '-show_format', '-show_streams',
            self.input_path
        ]
        result = subprocess.run(cmd, capture_output=True, text=True)
        return json.loads(result.stdout)

    def transcode(self, profile: TranscodeProfile) -> str:
        """
        Transcode video to a specific resolution using FFmpeg.
        Generates HLS segments for adaptive bitrate streaming.
        """
        output_path = f"{self.output_dir}/{profile.name}"
        os.makedirs(output_path, exist_ok=True)

        cmd = [
            'ffmpeg', '-i', self.input_path,
            '-vf', f'scale={profile.resolution}',
            '-c:v', 'libx264',
            '-b:v', profile.video_bitrate,
            '-c:a', 'aac',
            '-b:a', profile.audio_bitrate,
            '-hls_time', '6',                    # 6-second segments
            '-hls_playlist_type', 'vod',
            '-hls_segment_filename', f'{output_path}/segment_%04d.ts',
            f'{output_path}/playlist.m3u8',
            '-y'
        ]

        subprocess.run(cmd, check=True, capture_output=True)
        return output_path

    def generate_thumbnails(self) -> list[str]:
        """Generate thumbnails at different timestamps."""
        thumbnails = []
        for i, time_offset in enumerate([1, 5, 10, 30]):
            output = f"{self.output_dir}/thumb_{i}.jpg"
            cmd = [
                'ffmpeg', '-i', self.input_path,
                '-ss', str(time_offset),
                '-vframes', '1',
                '-vf', 'scale=1280:720',
                output, '-y'
            ]
            try:
                subprocess.run(cmd, check=True, capture_output=True)
                thumbnails.append(output)
            except subprocess.CalledProcessError:
                pass
        return thumbnails

    def generate_master_playlist(self, profiles_processed: list[TranscodeProfile]) -> str:
        """
        Generate HLS master playlist for adaptive bitrate streaming.
        Client uses this to switch quality based on bandwidth.
        """
        master = "#EXTM3U\n#EXT-X-VERSION:3\n\n"

        for profile in profiles_processed:
            bandwidth = int(profile.video_bitrate.replace('k', '')) * 1000
            master += (
                f"#EXT-X-STREAM-INF:BANDWIDTH={bandwidth},"
                f"RESOLUTION={profile.resolution}\n"
                f"{profile.name}/playlist.m3u8\n\n"
            )
```

ARCHITECTURE

```
        master_path = f"{self.output_dir}/master.m3u8"
        with open(master_path, 'w') as f:
            f.write(master)
        return master_path

    def process(self):
        """Full processing pipeline."""
        print(f"[{self.video_id}] Starting processing...")

        # 1. Extract metadata
        metadata = self.extract_metadata()
        source_height = int(metadata['streams'][0].get('height', 1080))

        # 2. Filter profiles (don't upscale)
        applicable = [p for p in PROFILES if p.height <= source_height]

        # 3. Transcode in parallel
        with ThreadPoolExecutor(max_workers=3) as executor:
            futures = {
                executor.submit(self.transcode, profile): profile
                for profile in applicable
            }
            processed = []
            for future in futures:
                future.result()  # wait
                processed.append(futures[future])

        # 4. Generate thumbnails
        thumbnails = self.generate_thumbnails()

        # 5. Generate master playlist
        self.generate_master_playlist(processed)

        # 6. Upload all to S3
        self._upload_to_s3()

        # 7. Update DB status → 'live'
        print(f"[{self.video_id}] Processing complete!")
        return {
            "video_id": self.video_id,
            "resolutions": [p.name for p in processed],
            "thumbnails": len(thumbnails),
            "status": "live"
        }

    def _upload_to_s3(self):
        """Upload all processed files to S3."""
        for root, dirs, files in os.walk(self.output_dir):
            for file in files:
                local_path = os.path.join(root, file)
                s3_key = f"processed/{self.video_id}/{os.path.relpath(local_path, self.output_dir)}"
                s3.upload_file(local_path, BUCKET_PROCESSED, s3_key)


# ■■■ Kafka Consumer (Processing Worker) ■■■
def processing_worker():
    consumer = Consumer({
        'bootstrap.servers': 'kafka:9092',
        'group.id': 'video-processors',
        'auto.offset.reset': 'earliest'
    })
    consumer.subscribe(['video-processing'])

    while True:
        msg = consumer.poll(1.0)
        if msg is None:
            continue

        event = json.loads(msg.value())
```

ARCHITECTURE

```
        video_id = event['video_id']

        # Download from S3
        local_path = f"/tmp/raw/{video_id}"
        s3.download_file(BUCKET_RAW, f"raw/{video_id}/original", local_path)

        # Process
        processor = VideoProcessor(video_id, local_path)
        result = processor.process()

        # Notify completion
        kafka_producer.produce(
            'video-ready',
            key=video_id.encode(),
            value=json.dumps(result).encode()
        )
```

Video Streaming (Adaptive Bitrate)

PYTHON

```python
"""
Streaming Flow:
  1. Client requests video → gets master.m3u8 from CDN
  2. Client's player picks appropriate quality playlist
  3. Player fetches .ts segments sequentially
  4. Player monitors bandwidth → switches quality dynamically
"""

from fastapi import FastAPI, Request
from fastapi.responses import RedirectResponse, StreamingResponse
import redis

app = FastAPI()
redis_client = redis.Redis(host='redis', port=6379)

@app.get("/api/v1/videos/{video_id}/stream")
async def get_stream_url(video_id: str, request: Request):
    """
    Returns the CDN URL for the master playlist.
    CDN selection is based on client's geographic location.
    """
    # Determine closest CDN PoP based on client IP
    client_ip = request.client.host
    cdn_node = select_closest_cdn(client_ip)

    # Generate signed URL (time-limited for security)
    signed_url = generate_signed_cdn_url(
        cdn_node=cdn_node,
        path=f"/processed/{video_id}/master.m3u8",
        expiry_seconds=3600
    )

    # Increment view count asynchronously
    redis_client.incr(f"views:{video_id}")

    return {"stream_url": signed_url, "cdn_node": cdn_node}
```

PYTHON

```python
# ■■■ View Count Aggregation (Batch) ■■■
async def flush_view_counts():
    """
    Periodically flush view counts from Redis to DB.
    Avoids write-heavy load on DB for every view.

    Runs every 30 seconds via a cron/scheduler.
    """
    cursor = 0
    while True:
        cursor, keys = redis_client.scan(cursor, match="views:*", count=100)
        for key in keys:
            video_id = key.decode().split(":")[1]
            count = int(redis_client.getdel(key) or 0)
            if count > 0:
                # Batch update DB
                await execute_sql(
                    "UPDATE videos SET view_count = view_count + %s WHERE video_id = %s",
                    (count, video_id)
                )
        if cursor == 0:
            break


def select_closest_cdn(client_ip: str) -> str:
    """
    Use GeoIP to find closest CDN Point of Presence.
    In production: DNS-based (like Route53 latency routing) or Anycast.
    """
    # Simplified: map IP ranges to regions
    cdn_nodes = {
        "us-east": "cdn-us-east.youtube.example.com",
        "us-west": "cdn-us-west.youtube.example.com",
        "eu-west": "cdn-eu-west.youtube.example.com",
        "ap-south": "cdn-ap-south.youtube.example.com",
    }
    region = geoip_lookup(client_ip)  # returns region key
    return cdn_nodes.get(region, cdn_nodes["us-east"])
```

Search & Recommendation

ARCHITECTURE

```
"""
Search: Elasticsearch for full-text search on title, description, tags.
Recommendations: Collaborative filtering + content-based.
"""

from elasticsearch import Elasticsearch

es = Elasticsearch(["http://elasticsearch:9200"])

# ■■■ Index a video ■■■
def index_video(video_id: str, title: str, description: str,
                tags: list, category: str, channel: str):
    es.index(index="videos", id=video_id, document={
        "title": title,
```

ARCHITECTURE

```
        "description": description,
```

ARCHITECTURE

```
        "tags": tags,
        "category": category,
        "channel": channel,
        "upload_date": "2024-01-15",
        "view_count": 0,
        "like_count": 0,
    })

# ■■■ Search videos ■■■
def search_videos(query: str, page: int = 0, size: int = 20) -> list:
    result = es.search(index="videos", body={
        "query": {
            "multi_match": {
                "query": query,
                "fields": ["title^3", "tags^2", "description", "channel"],
                "type": "best_fields",
                "fuzziness": "AUTO"
            }
        },
        "sort": [
            {"_score": "desc"},
            {"view_count": "desc"}
        ],
        "from": page * size,
        "size": size
    })
    return [hit["_source"] | {"video_id": hit["_id"]} for hit in result["hits"]["hits"]]


# ■■■ Simple Recommendation Engine ■■■
class RecommendationEngine:
    """
    Hybrid approach:
      1. Content-based: similar videos by tags/category
      2. Collaborative: users who watched X also watched Y
      3. Trending: popular videos in user's region
    """

    def get_recommendations(self, user_id: int, current_video_id: str,
                            limit: int = 20) -> list:
        # Weighted merge of different strategies
        content_based = self._content_based(current_video_id, limit=10)
        collaborative = self._collaborative(user_id, limit=10)
        trending = self._trending(limit=5)

        # Merge, deduplicate, rank
        seen = set()
        results = []
        for video in content_based + collaborative + trending:
            if video["video_id"] not in seen:
                seen.add(video["video_id"])
                results.append(video)

        return results[:limit]

    def _content_based(self, video_id: str, limit: int) -> list:
        """Find similar videos using Elasticsearch more-like-this."""
        result = es.search(index="videos", body={
            "query": {
                "more_like_this": {
                    "fields": ["title", "tags", "category"],
                    "like": [{"_index": "videos", "_id": video_id}],
                    "min_term_freq": 1,
                    "min_doc_freq": 1
                }
            },
            "size": limit
        })
        return [hit["_source"] for hit in result["hits"]["hits"]]
```

ARCHITECTURE

```
    def _collaborative(self, user_id: int, limit: int) -> list:
        """Users who watched similar videos also watched..."""
        watched = get_user_watch_history(user_id, limit=50)
        similar_users = get_similar_users(user_id, limit=20)
        candidates = get_unwatched_popular_among(similar_users, watched, limit)
        return candidates

    def _trending(self, limit: int) -> list:
        """Videos trending in the last 24 hours."""
        return redis_client.zrevrange("trending:global", 0, limit - 1)
```

9.6 Architecture Summary

SUMMARY

```
Key Design Decisions:
■■■
```

SYS 10
#10

System 10: Design WhatsApp

10.1 Requirements

REQUIREMENTS

```
Functional:
  • 1:1 messaging (text, media)
  • Group messaging (up to 1024 members)
  • Online/offline status & last seen
  • Message delivery receipts (sent ✓, delivered ✓✓, read ✓✓)
  • End-to-end encryption
  • Media sharing (images, videos, documents)
  • Push notifications for offline users

Non-Functional:
  • Ultra-low latency (<100ms for message delivery)
  • Exactly-once delivery semantics
  • 2B+ users, 100B+ messages/day
  • Messages stored only until delivered (or 30 days)
  • End-to-end encrypted
```

10.2 Capacity Estimation

CAPACITY

```
Total users:           2 Billion
DAU:                   500 Million
Messages/day:          100 Billion → ~1.15M messages/sec
Avg message size:      100 bytes (text)
Storage/day (text):    100B × 100B = 10 TB/day
Media messages:        ~5% = 5B/day, avg 200KB → 1 PB/day
Connections:           500M concurrent WebSocket connections
```

10.3 High-Level Architecture

ARCHITECTURE

```
┌──────────────┐        ┌──────────────┐
│  Client A  │        │  Client B  │
│  (Mobile)  │        │  (Mobile)  │
└──────────────┘        └──────────────┘
       │ WebSocket           │ WebSocket
       │                     │
       ▼                     ▼
```

ARCHITECTURE

```
■■■
```

```python
Partitioned by conversation for efficient range queries.

CREATE TABLE messages (
    conversation_id  TEXT,        -- sorted(user_a, user_b) hash for 1:1
    message_id       TIMEUUID,   -- TimeUUID for ordering
    sender_id        BIGINT,
    message_type     TEXT,        -- 'text', 'image', 'video', 'audio', 'document'
    content          BLOB,        -- encrypted message content
    media_url        TEXT,        -- S3 URL for media (encrypted)
    status           TEXT,        -- 'sent', 'delivered', 'read'
    created_at       TIMESTAMP,
    PRIMARY KEY (conversation_id, message_id)
) WITH CLUSTERING ORDER BY (message_id DESC)
  AND default_time_to_live = 2592000;  -- 30 day TTL

■■ MySQL: Users ■■
CREATE TABLE users (
    user_id          BIGINT PRIMARY KEY AUTO_INCREMENT,
    phone_number     VARCHAR(20) UNIQUE,
    display_name     VARCHAR(100),
    profile_pic_url  VARCHAR(500),
    public_key       BLOB,           -- for E2E encryption
    last_seen        DATETIME,
    status_text      VARCHAR(200),
    created_at       DATETIME
);

■■ MySQL: Groups ■■
CREATE TABLE groups (
    group_id         BIGINT PRIMARY KEY,
    group_name       VARCHAR(200),
    created_by       BIGINT,
    created_at       DATETIME,
    max_members      INT DEFAULT 1024
);

CREATE TABLE group_members (
    group_id         BIGINT,
    user_id          BIGINT,
    role             ENUM('admin', 'member'),
    joined_at        DATETIME,
    PRIMARY KEY (group_id, user_id)
);
"""
```

10.5 Core Components — Python Implementation

WebSocket Chat Server

```python
import asyncio
```

PYTHON

```python
import json
import uuid
import time
from datetime import datetime
from dataclasses import dataclass, field
from typing import Optional
from fastapi import FastAPI, WebSocket, WebSocketDisconnect
import aioredis
from cassandra.cluster import Cluster
from cassandra.query import SimpleStatement

app = FastAPI()

# ■■■ Connection Manager ■■■
class ConnectionManager:
    """
    Manages WebSocket connections on THIS server instance.
    Each server handles ~100K concurrent connections.
    """

    def __init__(self):
        self.active_connections: dict[int, WebSocket] = {}  # user_id → ws
        self.redis: Optional[aioredis.Redis] = None

    async def init_redis(self):
        self.redis = await aioredis.from_url("redis://redis-cluster:6379")

    async def connect(self, user_id: int, websocket: WebSocket):
        await websocket.accept()
        self.active_connections[user_id] = websocket

        # Register in Redis: which server handles this user
        server_id = os.environ.get("SERVER_ID", "chat-server-1")
        await self.redis.hset("user:sessions", str(user_id), server_id)
        await self.redis.set(f"user:online:{user_id}", "1")

        # Subscribe to this user's Redis channel (for cross-server messages)
        pubsub = self.redis.pubsub()
        await pubsub.subscribe(f"user:{user_id}:messages")
        asyncio.create_task(self._listen_redis(user_id, pubsub))

        # Deliver any pending offline messages
        await self._deliver_offline_messages(user_id)

        print(f"User {user_id} connected on {server_id}")

    async def disconnect(self, user_id: int):
        self.active_connections.pop(user_id, None)
        await self.redis.hdel("user:sessions", str(user_id))
        await self.redis.delete(f"user:online:{user_id}")

        # Update last_seen
        await self.redis.set(f"user:lastseen:{user_id}", str(int(time.time())))

    async def send_to_user(self, user_id: int, message: dict):
        """
        Send message to a user.
```

PYTHON

```python
        If user is on THIS server → direct WebSocket send.
        If user is on ANOTHER server → publish via Redis Pub/Sub.
        If user is OFFLINE → store in Kafka for offline delivery + push notification.
        """
        # Check if user is on this server
        if user_id in self.active_connections:
            ws = self.active_connections[user_id]
            await ws.send_json(message)
            return "delivered"

        # Check if user is online on another server
        target_server = await self.redis.hget("user:sessions", str(user_id))
        if target_server:
            # Publish to Redis Pub/Sub → other server picks it up
            await self.redis.publish(
                f"user:{user_id}:messages",
                json.dumps(message)
            )
            return "delivered"

        # User is offline → queue message
        await self._store_offline_message(user_id, message)
        await self._send_push_notification(user_id, message)
        return "stored"

    async def _listen_redis(self, user_id: int, pubsub):
        """Listen for messages from other servers via Redis Pub/Sub."""
        async for msg in pubsub.listen():
            if msg["type"] == "message":
                data = json.loads(msg["data"])
                if user_id in self.active_connections:
                    await self.active_connections[user_id].send_json(data)

    async def _store_offline_message(self, user_id: int, message: dict):
        """Store message in sorted set for offline delivery."""
        await self.redis.zadd(
            f"offline:{user_id}",
            {json.dumps(message): message["timestamp"]}
        )

    async def _deliver_offline_messages(self, user_id: int):
        """Deliver all pending offline messages when user connects."""
        messages = await self.redis.zrangebyscore(
            f"offline:{user_id}", "-inf", "+inf"
        )
        for msg_data in messages:
            message = json.loads(msg_data)
            await self.active_connections[user_id].send_json(message)

        # Clear delivered messages
        if messages:
            await self.redis.delete(f"offline:{user_id}")

    async def _send_push_notification(self, user_id: int, message: dict):
        """Send push notification via APNs/FCM."""
        await self.redis.lpush("push:queue", json.dumps({
            "user_id": user_id,
```

PYTHON

```python
            "title": message.get("sender_name", "New Message"),
            "body": message.get("preview", "You have a new message"),
            "data": {"conversation_id": message.get("conversation_id")}
        }))


manager = ConnectionManager()

@app.on_event("startup")
async def startup():
    await manager.init_redis()


# ■■■ WebSocket Endpoint ■■■
@app.websocket("/ws/{user_id}")
async def websocket_endpoint(websocket: WebSocket, user_id: int):
    await manager.connect(user_id, websocket)

    try:
        while True:
            data = await websocket.receive_json()
            await handle_message(user_id, data)
    except WebSocketDisconnect:
        await manager.disconnect(user_id)


async def handle_message(sender_id: int, data: dict):
    """Route incoming messages based on type."""
    msg_type = data.get("type")

    if msg_type == "text_message":
        await handle_text_message(sender_id, data)
    elif msg_type == "group_message":
        await handle_group_message(sender_id, data)
    elif msg_type == "ack":
        await handle_ack(sender_id, data)
    elif msg_type == "typing":
        await handle_typing_indicator(sender_id, data)
    elif msg_type == "read_receipt":
        await handle_read_receipt(sender_id, data)
```

Message Handling with Delivery Guarantees

SUMMARY

```
"""
Message Delivery Flow:
■■■■■■■■■■■■■■■■■■■■■■■■
 Client A                Server              Client B
    ■                       ■                   ■
    ■■■■■ Send Message ■■■■■■                     ■
    ■                       ■■■ Store in DB ■■■  ■
    ■■■■■■ ACK (sent) ■■■■■■                      ■

    ■                       ■                   ■
```

```
    ■                       ■■■ Deliver ■■■■■■■■■■
    ■                       ■■■■ ACK (received) ■■
    ■■■■ Delivered ■■■■■■■■■■                    ■
    ■                       ■                   ■
    ■                       ■    [User opens]   ■
    ■                       ■■■■ Read receipt ■■■■
    ■■■■■■ Read ■■■■■■■■■■■■■                    ■
"""

from cassandra.cluster import Cluster
from cassandra.util import uuid_from_time

# Cassandra connection
cassandra = Cluster(['cassandra-node1', 'cassandra-node2'])
session = cassandra.connect('whatsapp')

# Prepared statements for performance
INSERT_MSG = session.prepare("""
    INSERT INTO messages (conversation_id, message_id, sender_id,
                          message_type, content, status, created_at)
    VALUES (?, ?, ?, ?, ?, ?, ?)
""")

UPDATE_STATUS = session.prepare("""
    UPDATE messages SET status = ?
    WHERE conversation_id = ? AND message_id = ?
""")


def get_conversation_id(user_a: int, user_b: int) -> str:
    """Deterministic conversation ID for 1:1 chats."""
    return f"conv:{min(user_a, user_b)}:{max(user_a, user_b)}"


async def handle_text_message(sender_id: int, data: dict):
    """Process a 1:1 text message with exactly-once delivery."""
    recipient_id = data["recipient_id"]
    content = data["content"]
    client_msg_id = data["client_message_id"]  # Client-generated for idempotency

    conversation_id = get_conversation_id(sender_id, recipient_id)
    message_id = uuid_from_time(time.time())  # TimeUUID

    # ■■■ 1. Persist to Cassandra ■■■
    session.execute(INSERT_MSG, (
        conversation_id, message_id, sender_id,
        'text', content.encode(), 'sent', datetime.utcnow()
    ))

    # ■■■ 2. ACK to sender (message stored → single tick) ■■■
    await manager.send_to_user(sender_id, {
        "type": "ack",
        "client_message_id": client_msg_id,
        "message_id": str(message_id),
        "status": "sent",
        "timestamp": int(time.time())
```

```
    })

    # ■■■ 3. Deliver to recipient ■■■
    message_payload = {
        "type": "text_message",
        "message_id": str(message_id),
        "conversation_id": conversation_id,
        "sender_id": sender_id,
        "content": content,
        "timestamp": int(time.time())
    }

    delivery_status = await manager.send_to_user(recipient_id, message_payload)

    # ■■■ 4. If delivered, notify sender (double tick) ■■■
    if delivery_status == "delivered":
        session.execute(UPDATE_STATUS, ('delivered', conversation_id, message_id))
        await manager.send_to_user(sender_id, {
            "type": "status_update",
            "message_id": str(message_id),
            "status": "delivered"
        })


async def handle_read_receipt(reader_id: int, data: dict):
    """User has read messages → notify sender."""
    conversation_id = data["conversation_id"]
    last_read_message_id = data["last_read_message_id"]

    session.execute(UPDATE_STATUS, ('read', conversation_id,
                                    uuid.UUID(last_read_message_id)))

    # Notify the other party
    other_user_id = data["sender_id"]
    await manager.send_to_user(other_user_id, {
        "type": "read_receipt",
        "conversation_id": conversation_id,
        "last_read_message_id": last_read_message_id,
        "reader_id": reader_id
    })


async def handle_group_message(sender_id: int, data: dict):
    """
    Group message: fan-out to all group members.
    For large groups, use async fan-out via message queue.
    """
    group_id = data["group_id"]
    content = data["content"]

    conversation_id = f"group:{group_id}"
    message_id = uuid_from_time(time.time())

    # Persist message once
    session.execute(INSERT_MSG, (
        conversation_id, message_id, sender_id,
```

```
        'text', content.encode(), 'sent', datetime.utcnow()
    ))

    # Get group members
    members = await get_group_members(group_id)

    message_payload = {
        "type": "group_message",
        "message_id": str(message_id),
        "group_id": group_id,
        "sender_id": sender_id,
        "content": content,
        "timestamp": int(time.time())
    }

    # Fan-out to all members (except sender)
    tasks = []
    for member_id in members:
        if member_id != sender_id:
            tasks.append(manager.send_to_user(member_id, message_payload))

    await asyncio.gather(*tasks)
```

End-to-End Encryption (Signal Protocol Simplified)

PYTHON

```python
"""
WhatsApp uses the Signal Protocol for E2E encryption.
Simplified implementation of the key concepts.
"""

from cryptography.hazmat.primitives.asymmetric.x25519 import (
    X25519PrivateKey, X25519PublicKey
)
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
import os

class E2EEncryption:
    """
    Simplified Signal Protocol:
    1. Each user generates an identity key pair
    2. Key exchange happens on first message (X3DH)
    3. Messages encrypted with derived symmetric key
    4. Server NEVER sees plaintext
    """

    def __init__(self):
        # Long-term identity key
        self.identity_key = X25519PrivateKey.generate()
        self.identity_public = self.identity_key.public_key()
```

PYTHON

```python
        # Ephemeral keys for each session
        self.sessions: dict[int, bytes] = {}  # peer_id → shared_secret

    def get_public_key_bytes(self) -> bytes:
        """Public key to share with server for key distribution."""
        from cryptography.hazmat.primitives.serialization import (
            Encoding, PublicFormat
        )
        return self.identity_public.public_bytes(
            Encoding.Raw, PublicFormat.Raw
        )

    def establish_session(self, peer_id: int, peer_public_key_bytes: bytes):
        """
        Perform key exchange (simplified X3DH → Diffie-Hellman).
        In production: includes prekeys, signed prekeys, etc.
        """
        peer_public_key = X25519PublicKey.from_public_bytes(peer_public_key_bytes)

        # Diffie-Hellman key exchange
        shared_secret = self.identity_key.exchange(peer_public_key)

        # Derive symmetric key using HKDF
        derived_key = HKDF(
            algorithm=hashes.SHA256(),
            length=32,
            salt=None,
            info=b"whatsapp-e2e"
        ).derive(shared_secret)

        self.sessions[peer_id] = derived_key
        return derived_key

    def encrypt_message(self, peer_id: int, plaintext: str) -> tuple[bytes, bytes]:
        """Encrypt a message for a peer."""
        key = self.sessions[peer_id]
        nonce = os.urandom(12)  # 96-bit nonce

        aesgcm = AESGCM(key)
        ciphertext = aesgcm.encrypt(nonce, plaintext.encode(), None)

        return nonce, ciphertext

    def decrypt_message(self, peer_id: int, nonce: bytes,
                        ciphertext: bytes) -> str:
        """Decrypt a message from a peer."""
        key = self.sessions[peer_id]

        aesgcm = AESGCM(key)
        plaintext = aesgcm.decrypt(nonce, ciphertext, None)

        return plaintext.decode()


# ■■■ Usage Example ■■■
def demo_e2e():
    alice = E2EEncryption()
```

```python
bob = E2EEncryption()

# Exchange public keys (via server)
alice.establish_session(peer_id=2, peer_public_key_bytes=bob.get_public_key_bytes())
bob.establish_session(peer_id=1, peer_public_key_bytes=alice.get_public_key_bytes())

# Alice encrypts
nonce, ciphertext = alice.encrypt_message(peer_id=2, plaintext="Hello Bob!")

# Bob decrypts
plaintext = bob.decrypt_message(peer_id=1, nonce=nonce, ciphertext=ciphertext)
print(f"Decrypted: {plaintext}")  # "Hello Bob!"

# Server only sees ciphertext — cannot read the message!
```

Presence (Online/Offline/Last Seen)

```python
"""
Presence Service:
  • Track online/offline status
  • Last seen timestamp
  • Typing indicators

  Uses Redis with heartbeat mechanism.
"""

class PresenceService:
    def __init__(self, redis_client):
        self.redis = redis_client
        self.HEARTBEAT_INTERVAL = 30    # seconds
        self.ONLINE_TIMEOUT = 45        # seconds

    async def heartbeat(self, user_id: int):
        """
        Client sends heartbeat every 30s.
        If no heartbeat for 45s → user is offline.
        """
        await self.redis.setex(
            f"presence:{user_id}",
            self.ONLINE_TIMEOUT,
            "online"
        )
        await self.redis.set(
            f"lastseen:{user_id}",
            str(int(time.time()))
        )

    async def is_online(self, user_id: int) -> bool:
        status = await self.redis.get(f"presence:{user_id}")
        return status == b"online"

    async def get_last_seen(self, user_id: int) -> Optional[int]:
        ts = await self.redis.get(f"lastseen:{user_id}")
```

```python
        return int(ts) if ts else None

    async def get_bulk_presence(self, user_ids: list[int]) -> dict:
        """
        Get online status for multiple users (e.g., contact list).
        Uses Redis pipeline for efficiency.
        """
        pipe = self.redis.pipeline()
        for uid in user_ids:
            pipe.get(f"presence:{uid}")

        results = await pipe.execute()
        return {
            uid: (result == b"online")
            for uid, result in zip(user_ids, results)
        }

    async def send_typing_indicator(self, sender_id: int,
                                    recipient_id: int, is_typing: bool):
        """Ephemeral typing indicator — not persisted."""
        await manager.send_to_user(recipient_id, {
            "type": "typing",
            "user_id": sender_id,
            "is_typing": is_typing
        })
```

10.6 Architecture Summary

```summary
Key Design Decisions:
■■■
```

SYS 11
#11

System 11: Design Instagram Feed

11.1 Requirements

REQUIREMENTS

```
Functional:
  • Post photos/videos with captions
  • Follow/unfollow users
  • News feed: see posts from people you follow
  • Like, comment on posts
  • Stories (24-hour ephemeral content)
  • Explore/discover page

Non-Functional:
  • Feed generation < 500ms
  • 500M DAU
  • 100M photos uploaded/day
  • Feed should be ranked (not purely chronological)
  • High availability, eventual consistency OK for feed
```

11.2 Capacity Estimation

CAPACITY

```
DAU:                    500 Million
Posts/day:              100 Million
Avg photo size:         2 MB
Storage/day (photos):   100M × 2MB = 200 TB
Avg follows per user:   200
Feed requests/day:      500M × 10 opens = 5B
Feed generation:        5B / 86400 ≈ 58K requests/sec
```

11.3 High-Level Architecture

ARCHITECTURE

```
┌──────────────────────┐
│      Clients         │
│  (iOS/Android/Web)   │
└──────────────────────┘
          │
          ▼
┌──────────────────────────────────────────────────────────────┐
│                      API Gateway                             │
```

ARCHITECTURE

```
■           (Auth, Rate Limiting, Routing)                    ■
■■■
```

ARCHITECTURE

```
    media_id        BIGINT PRIMARY KEY,
    post_id         BIGINT,
    media_type      ENUM('photo', 'video', 'carousel'),
    media_url       VARCHAR(500),       -- CDN URL
    width           INT,
    height          INT,
    display_order   TINYINT,
    INDEX idx_post (post_id)
);

■■ Social Graph ■■
CREATE TABLE follows (
    follower_id     BIGINT,
    followee_id     BIGINT,
    created_at      DATETIME,
    PRIMARY KEY (follower_id, followee_id),
    INDEX idx_followee (followee_id, follower_id)
);
-- Shard by follower_id for "who do I follow" queries
-- Secondary index on followee_id for "who follows me"

■■ Redis: Pre-computed Feed ■■
-- Key: feed:{user_id}
-- Type: Sorted Set (score = timestamp)
-- Value: post_id
-- Max size: 1000 posts
-- TTL: 7 days

■■ Likes ■■
CREATE TABLE likes (
    user_id         BIGINT,
    post_id         BIGINT,
    created_at      DATETIME,
    PRIMARY KEY (user_id, post_id)
);
"""
```

11.5 Core Implementation

Post Creation & Fan-Out

ARCHITECTURE

```
import asyncio
import time
from dataclasses import dataclass
from typing import Optional
import redis.asyncio as aioredis
from confluent_kafka import Producer
import json

# ■■■ Snowflake ID Generator ■■■
class SnowflakeIDGenerator:
    """
    Twitter Snowflake-like ID generator.
    64 bits: [1 bit unused][41 bits timestamp][10 bits machine][12 bits sequence]
    """

    EPOCH = 1640995200000  # 2022-01-01 00:00:00 UTC in ms

    def __init__(self, machine_id: int):
        self.machine_id = machine_id & 0x3FF  # 10 bits
        self.sequence = 0
```

ARCHITECTURE

```
        self.last_timestamp = -1

    def generate(self) -> int:
        timestamp = int(time.time() * 1000) - self.EPOCH

        if timestamp == self.last_timestamp:
            self.sequence = (self.sequence + 1) & 0xFFF  # 12 bits
            if self.sequence == 0:
                # Wait for next millisecond
                while timestamp <= self.last_timestamp:
                    timestamp = int(time.time() * 1000) - self.EPOCH
        else:
            self.sequence = 0

        self.last_timestamp = timestamp

        return (
            (timestamp << 22) |
            (self.machine_id << 12) |
            self.sequence
        )

id_gen = SnowflakeIDGenerator(machine_id=1)
redis_client = aioredis.from_url("redis://redis-cluster:6379")
kafka_producer = Producer({'bootstrap.servers': 'kafka:9092'})

CELEBRITY_THRESHOLD = 10_000  # followers


# ■■■ Post Creation ■■■
async def create_post(user_id: int, caption: str,
                      media_urls: list[str]) -> dict:
    """Create a new post and trigger fan-out."""
    post_id = id_gen.generate()
    created_at = time.time()

    # 1. Store post in DB
    await execute_sql(
        "INSERT INTO posts (post_id, user_id, caption, created_at) VALUES (%s,%s,%s,%s)",
        (post_id, user_id, caption, created_at)
    )

    # 2. Store media
    for i, url in enumerate(media_urls):
        media_id = id_gen.generate()
        await execute_sql(
            "INSERT INTO post_media (media_id, post_id, media_url, display_order) "
            "VALUES (%s,%s,%s,%s)",
            (media_id, post_id, url, i)
        )

    # 3. Trigger fan-out
    follower_count = await get_follower_count(user_id)

    if follower_count < CELEBRITY_THRESHOLD:
        # Fan-out on write: push to all followers' feeds
        kafka_producer.produce(
            'feed-fanout',
            key=str(user_id).encode(),
            value=json.dumps({
                "type": "fanout_write",
                "post_id": post_id,
                "user_id": user_id,
                "created_at": created_at
            }).encode()
        )
    else:
        # Celebrity: store in celebrity posts cache only
        await redis_client.zadd(
```

ARCHITECTURE

```
            f"celebrity:posts:{user_id}",
            {str(post_id): created_at}
        )
        # Trim to last 100 posts
        await redis_client.zremrangebyrank(
            f"celebrity:posts:{user_id}", 0, -101
        )

    kafka_producer.flush()

    return {"post_id": post_id, "status": "created"}


# ■■■ Fan-Out Worker (Kafka Consumer) ■■■
class FanOutWorker:
    """
    Consumes from 'feed-fanout' topic.
    Pushes post_id to each follower's feed cache in Redis.

    For a user with 500 followers → 500 Redis ZADD operations.
    """

    FEED_MAX_SIZE = 1000      # Keep last 1000 posts in feed cache
    BATCH_SIZE = 500           # Process followers in batches

    async def process_fanout(self, event: dict):
        post_id = event["post_id"]
        user_id = event["user_id"]
        created_at = event["created_at"]

        # Get all followers (paginated)
        cursor = 0
        while True:
            followers = await get_followers_batch(
                user_id, cursor, self.BATCH_SIZE
            )
            if not followers:
                break

            # Batch Redis operations using pipeline
            pipe = redis_client.pipeline()
            for follower_id in followers:
                pipe.zadd(
                    f"feed:{follower_id}",
                    {str(post_id): created_at}
                )
                # Trim feed to max size
                pipe.zremrangebyrank(
                    f"feed:{follower_id}", 0, -(self.FEED_MAX_SIZE + 1)
                )

            await pipe.execute()
            cursor += self.BATCH_SIZE

        print(f"Fan-out complete for post {post_id} by user {user_id}")
```

Feed Generation (Hybrid Pull + Push)

SUMMARY

```
"""
Feed Generation: Hybrid Approach
■■■
```

```
1. Fetch pre-computed feed from Redis (pushed by non-celebrities)
2. Fetch latest posts from followed celebrities (pulled on-read)
3. Merge and rank
4. Return paginated results
"""

@dataclass
class FeedItem:
    post_id: int
    user_id: int
    caption: str
    media_urls: list[str]
    created_at: float
    like_count: int
    comment_count: int
    user_name: str
    user_avatar: str
    ranking_score: float = 0.0


class FeedService:
    def __init__(self):
        self.redis = redis_client
        self.ranking_model = FeedRankingModel()

    async def get_feed(self, user_id: int, page: int = 0,
                       page_size: int = 20) -> list[FeedItem]:
        """
        Generate user's feed with hybrid fan-out approach.
        """

        # ■■■ Step 1: Pre-computed feed (fan-out on write results) ■■■
        cached_post_ids = await self.redis.zrevrange(
            f"feed:{user_id}",
            0, 500, withscores=True
        )

        post_scores = {
            int(pid): score
            for pid, score in cached_post_ids
        }

        # ■■■ Step 2: Celebrity posts (fan-out on read) ■■■
        followed_celebrities = await self._get_followed_celebrities(user_id)

        if followed_celebrities:
            pipe = self.redis.pipeline()
            for celeb_id in followed_celebrities:
                pipe.zrevrange(
                    f"celebrity:posts:{celeb_id}",
                    0, 20, withscores=True
                )

            celeb_results = await pipe.execute()

            for posts_with_scores in celeb_results:
```

```
                for pid, score in posts_with_scores:
                    post_scores[int(pid)] = score

        # ■■■ Step 3: Get full post data ■■■
        all_post_ids = sorted(post_scores.keys(),
                              key=lambda x: post_scores[x],
                              reverse=True)[:200]

        posts = await self._batch_get_posts(all_post_ids)

        # ■■■ Step 4: Rank posts ■■■
        feed_items = await self._enrich_posts(posts, user_id)
        ranked_feed = self.ranking_model.rank(user_id, feed_items)

        # ■■■ Step 5: Paginate ■■■
        start = page * page_size
        end = start + page_size

        return ranked_feed[start:end]


# ■■■ Feed Ranking Model ■■■
class FeedRankingModel:
    """
    ML-based ranking considering:
    - Recency (time decay)
    - User affinity (how often you interact with poster)
    - Engagement (likes/comments ratio)
    - Content type preference
    - Post quality signals
    """

    def rank(self, user_id: int, items: list[FeedItem]) -> list[FeedItem]:
        """Score and sort feed items."""
        for item in items:
            item.ranking_score = self._compute_score(user_id, item)

        return sorted(items, key=lambda x: x.ranking_score, reverse=True)

    def _compute_score(self, user_id: int, item: FeedItem) -> float:
        """
        Score = w1 * affinity + w2 * recency + w3 * engagement + w4 * quality
        """
        # Time decay: exponential decay over hours
        age_hours = (time.time() - item.created_at) / 3600
        recency_score = 1.0 / (1.0 + age_hours * 0.1)

        # Engagement score
        total_interactions = item.like_count + item.comment_count * 2
        engagement_score = min(total_interactions / 100.0, 1.0)

        # User affinity (precomputed: how often user_id interacts with poster)
        affinity = self._get_affinity(user_id, item.user_id)

        # Weighted combination
        score = (
```

```
            0.35 * affinity +
            0.30 * recency_score +
            0.25 * engagement_score +
            0.10 * 0.5  # content quality placeholder
        )

        return score

    def _get_affinity(self, user_id: int, poster_id: int) -> float:
        """
        User affinity: precomputed score based on interaction history.
        Cached in Redis, updated by batch job.
        """
        cached = redis_client.get(f"affinity:{user_id}:{poster_id}")
        return float(cached) if cached else 0.3  # default
```

Stories (Ephemeral Content)

```python
"""
Instagram Stories: 24-hour ephemeral content.
Design considerations:
  - TTL-based expiry
  - Separate from main feed
  - Ordered by recency & affinity
"""

class StoriesService:
    """
    Stories Architecture:
    - Stored in Redis with 24-hour TTL
    - Each user's stories: Redis Sorted Set
    - Story tray (top bar): precomputed list of users with active stories
    """

    STORY_TTL = 86400  # 24 hours in seconds

    async def create_story(self, user_id: int, media_url: str,
                           story_type: str = "photo") -> dict:
        story_id = id_gen.generate()
        created_at = time.time()

        story_data = {
            "story_id": story_id,
            "user_id": user_id,
            "media_url": media_url,
            "type": story_type,
            "created_at": created_at,
            "expires_at": created_at + self.STORY_TTL
        }

        # Store story with TTL
        await self.redis.setex(
            f"story:{story_id}",
```

PYTHON

```python
            self.STORY_TTL,
            json.dumps(story_data)
        )

        # Add to user's story list
        await self.redis.zadd(
            f"user:{user_id}:stories",
            {str(story_id): created_at}
        )
        await self.redis.expire(f"user:{user_id}:stories", self.STORY_TTL)

        # Update story tray for followers
        followers = await get_all_followers(user_id)
        pipe = self.redis.pipeline()
        for fid in followers:
            pipe.zadd(f"storytray:{fid}", {str(user_id): created_at})
            pipe.expire(f"storytray:{fid}", self.STORY_TTL)
        await pipe.execute()

        return story_data

    async def get_story_tray(self, user_id: int) -> list[dict]:
        """
        Get the story tray (horizontal scroll bar at top).
        Returns users who have active stories, ordered by affinity.
        """
        active_users = await self.redis.zrevrange(
            f"storytray:{user_id}", 0, 100, withscores=True
        )

        tray = []
        for uid_bytes, timestamp in active_users:
            poster_id = int(uid_bytes)
            user_info = await get_user_info(poster_id)
            story_count = await self.redis.zcard(f"user:{poster_id}:stories")

            viewed_count = await self.redis.scard(
                f"viewed:{user_id}:{poster_id}"
            )

            tray.append({
                "user_id": poster_id,
                "username": user_info["username"],
                "avatar": user_info["profile_pic_url"],
                "story_count": story_count,
                "has_unseen": viewed_count < story_count,
                "latest_timestamp": timestamp
            })

        return tray
```

SYS 12
#12

System 12: Design Twitter Timeline

12.1 Requirements

REQUIREMENTS

```
Functional:
  • Post tweets (280 chars, media)
  • Home timeline (tweets from followed users)
  • User timeline (all tweets by a specific user)
  • Retweet, quote tweet
  • Like, reply
  • Search tweets
  • Trending topics
  • Follow/unfollow

Non-Functional:
  • Timeline generation < 300ms
  • 400M DAU, 500M tweets/day
  • Timeline should feel real-time
  • High read:write ratio (~1000:1)
  • Availability > consistency
```

12.2 High-Level Architecture

ARCHITECTURE

```
═══════════════════════════════════════════════════════════════════════
■                       CLIENTS                                     ■
■            Web   ■  iOS   ■  Android  ■  API                      ■
═══════════════════════════════════════════════════════════════════════
                            ■
                            ▼
═══════════════════════════════════════════════════════════════════════
■                     API GATEWAY                                    ■
■         (Authentication, Rate Limiting, Routing)                   ■
═══════════════════════════════════════════════════════════════════════
   ■         ■          ■          ■          ■
   ▼         ▼          ▼          ▼          ▼
═══════  ═══════  ═══════  ═══════  ═══════
■Tweet ■  ■Home   ■  ■Search  ■  ■User    ■  ■Trending   ■
■Write ■  ■Time-  ■  ■Service ■  ■Service ■  ■Service    ■
■Svc   ■  ■line   ■  ■       ■  ■       ■  ■           ■
■      ■  ■Svc    ■  ■(Elastic■  ■(Follow ■  ■(Count-Min ■
■      ■  ■       ■  ■Search) ■  ■Graph)  ■  ■ Sketch)   ■
═══════  ═══════  ═══════  ═══════  ═══════
   ■         ■
   ■         ■  HOME TIMELINE FLOW
   ■         ■  ═══════════════════
   ■         ■
   ■         ■  Is user's feed cached?
   ■         ■
```

ARCHITECTURE

```
   ■          ■  YES → Redis Feed Cache (Pre-built)
   ■          ■  NO  → Fan-out on Read (merge followed users' posts)
   ■
   ■  TWEET WRITE FLOW
   ■  ■■■■■■■■■■■■■■■■■■
   ■
   ▼
■■■■■■■■■■■■■■■■■■
```

ARCHITECTURE

```
CELEBRITY_THRESHOLD = 10_000

@dataclass
class Tweet:
    tweet_id: int
    user_id: int
    content: str
    created_at: float
    reply_to: Optional[int] = None
    retweet_of: Optional[int] = None
    quote_of: Optional[int] = None
    media_urls: list[str] = field(default_factory=list)
    hashtags: list[str] = field(default_factory=list)
    mentions: list[str] = field(default_factory=list)
    like_count: int = 0
    retweet_count: int = 0
    reply_count: int = 0


class TweetService:

    def __init__(self):
        self.id_gen = SnowflakeIDGenerator(machine_id=1)

    async def create_tweet(self, user_id: int, content: str,
                           media_urls: list[str] = None,
                           reply_to: int = None,
                           quote_of: int = None) -> Tweet:
        """Create a tweet and trigger downstream processing."""
        # ■■■ Validation ■■■
        if len(content) > TWEET_MAX_LENGTH:
            raise HTTPException(400, "Tweet exceeds 280 characters")

        # ■■■ Parse entities ■■■
        hashtags = re.findall(r'#(\w+)', content)
        mentions = re.findall(r'@(\w+)', content)

        tweet_id = self.id_gen.generate()
        created_at = time.time()

        tweet = Tweet(
            tweet_id=tweet_id,
            user_id=user_id,
            content=content,
            created_at=created_at,
            reply_to=reply_to,
            quote_of=quote_of,
            media_urls=media_urls or [],
            hashtags=hashtags,
            mentions=mentions
        )

        # ■■■ 1. Persist to DB ■■■
        await self._store_tweet(tweet)

        # ■■■ 2. Cache tweet data ■■■
        await redis_client.setex(
            f"tweet:{tweet_id}",
            86400,  # 24 hour cache
            json.dumps(tweet.__dict__, default=str)
        )

        # ■■■ 3. Add to user's own timeline ■■■
        await redis_client.zadd(
            f"user_timeline:{user_id}",
            {str(tweet_id): created_at}
        )
        await redis_client.zremrangebyrank(
            f"user_timeline:{user_id}", 0, -3201  # Keep last 3200
```

ARCHITECTURE

```
        )

        # ■■■ 4. Publish event for async processing ■■■
        event = {
            "event_type": "tweet_created",
            "tweet_id": tweet_id,
            "user_id": user_id,
            "content": content,
            "hashtags": hashtags,
            "mentions": mentions,
            "created_at": created_at,
            "follower_count": await self._get_follower_count(user_id)
        }

        kafka_producer.produce(
            'tweet-events',
            key=str(user_id).encode(),
            value=json.dumps(event).encode(),
            partition=user_id % 16  # partition by user
        )
        kafka_producer.flush()

        return tweet
```

Fan-Out Service (The Heart of Twitter)

SUMMARY

```
"""
Fan-Out Service: The Celebrity Problem
■■■
```

```
    async def _handle_tweet_fanout(self, event: dict):
        tweet_id = event['tweet_id']
        user_id = event['user_id']
        created_at = event['created_at']
        follower_count = event['follower_count']

        # ■■■ Decision: Fan-out on write or skip? ■■■
        if follower_count >= CELEBRITY_THRESHOLD:
            # Celebrity: mark as celebrity tweet, skip fan-out
            await redis_client.zadd(
                f"celebrity_tweets:{user_id}",
                {str(tweet_id): created_at}
            )
            await redis_client.zremrangebyrank(
                f"celebrity_tweets:{user_id}", 0, -201
            )
            return

        # ■■■ Regular user: push to all followers ■■■
        total_pushed = 0
        cursor = 0

        while True:
            followers = await self._get_followers_batch(
                user_id, cursor, self.BATCH_SIZE
            )

            if not followers:
                break

            # Pipeline Redis operations for efficiency
            pipe = redis_client.pipeline()
            for follower_id in followers:
                pipe.zadd(
                    f"home_timeline:{follower_id}",
                    {str(tweet_id): created_at}
                )
                pipe.zremrangebyrank(
                    f"home_timeline:{follower_id}",
                    0, -(self.FEED_CACHE_SIZE + 1)
                )

            await pipe.execute()

            total_pushed += len(followers)
            cursor += self.BATCH_SIZE

        print(f"Tweet {tweet_id}: fan-out to {total_pushed} followers")


# ■■■ Handle follow/unfollow ■■■
class SocialGraphService:

    async def follow(self, follower_id: int, followee_id: int):
        """User A follows User B. Backfill A's timeline with B's recent tweets."""
        # 1. Store in DB
```

```
        await execute_sql(
            "INSERT IGNORE INTO follows (follower_id, followee_id, created_at) "
            "VALUES (%s, %s, NOW())",
            (follower_id, followee_id)
        )

        # 2. Update Redis graph
        await redis_client.zadd(
            f"following:{follower_id}",
            {str(followee_id): time.time()}
        )
        await redis_client.zadd(
            f"followers:{followee_id}",
            {str(follower_id): time.time()}
        )

        # 3. Update follower count
        await redis_client.incr(f"follower_count:{followee_id}")

        # 4. Backfill: Add B's recent tweets to A's timeline
        follower_count = await redis_client.get(f"follower_count:{followee_id}")

        if int(follower_count or 0) < CELEBRITY_THRESHOLD:
            recent_tweets = await redis_client.zrevrange(
                f"user_timeline:{followee_id}", 0, 50, withscores=True
            )

            if recent_tweets:
                pipe = redis_client.pipeline()
                for tweet_id, score in recent_tweets:
                    pipe.zadd(
                        f"home_timeline:{follower_id}",
                        {tweet_id: score}
                    )
                await pipe.execute()
```

Home Timeline Service

ARCHITECTURE

```
"""
Home Timeline: Merge pre-computed feed with celebrity posts.

Timeline Read Path:
  1. Get pre-computed feed from Redis (pushed by fan-out workers)
  2. Get list of celebrities the user follows
  3. Fetch recent tweets from each celebrity
  4. Merge, deduplicate, sort by time
  5. Apply ranking (optional)
  6. Return paginated results
"""

class HomeTimelineService:

    CACHE_SIZE = 800

    async def get_timeline(self, user_id: int,
                           cursor: Optional[str] = None,
```

ARCHITECTURE

```
                    count: int = 20) -> dict:
    """
    Get home timeline with cursor-based pagination.
    Cursor is the tweet_id of the last seen tweet.
    """

    # ■■■ Step 1: Pre-computed feed (fan-out on write) ■■■
    if cursor:
        cursor_score = float(cursor)
        cached_tweets = await redis_client.zrevrangebyscore(
            f"home_timeline:{user_id}",
            f"({cursor_score}", "-inf",
            start=0, num=count + 50,
            withscores=True
        )
    else:
        cached_tweets = await redis_client.zrevrange(
            f"home_timeline:{user_id}",
            0, count + 50, withscores=True
        )

    timeline = [(int(tid), score) for tid, score in cached_tweets]

    # ■■■ Step 2: Celebrity tweets (fan-out on read) ■■■
    celeb_ids = await redis_client.smembers(
        f"user:{user_id}:celeb_follows"
    )

    if celeb_ids:
        pipe = redis_client.pipeline()
        for cid in celeb_ids:
            pipe.zrevrange(
                f"celebrity_tweets:{int(cid)}",
                0, 10, withscores=True
            )

        results = await pipe.execute()
        for celeb_tweets in results:
            for tid, score in celeb_tweets:
                timeline.append((int(tid), score))

    # ■■■ Step 3: Merge, deduplicate, sort ■■■
    seen = set()
    unique_timeline = []
    for tweet_id, score in timeline:
        if tweet_id not in seen:
            seen.add(tweet_id)
            unique_timeline.append((tweet_id, score))

    unique_timeline.sort(key=lambda x: x[1], reverse=True)
    page = unique_timeline[:count]

    # ■■■ Step 4: Hydrate tweets with full data ■■■
    tweet_ids = [tid for tid, _ in page]
    tweets = await self._hydrate_tweets(tweet_ids, user_id)

    # ■■■ Step 5: Build response with cursor ■■■
    next_cursor = None
    if page:
        next_cursor = str(page[-1][1])

    return {
        "tweets": tweets,
        "next_cursor": next_cursor,
        "has_more": len(unique_timeline) > count
    }
```

Trending Topics (Count-Min Sketch)

PYTHON

```python
"""
Trending Topics: Find trending hashtags in real-time.

Challenge: Counting millions of hashtags per minute accurately
           without storing every single one.

Solution: Count-Min Sketch (probabilistic data structure)
          + Time-windowed buckets
"""

import hashlib
import math
from collections import defaultdict

class CountMinSketch:
    """
    Probabilistic data structure for frequency estimation.

    Space: O(w x d) where w = width, d = depth
    Error: e = e/w (additive)
    Probability of exceeding error: d = e^(-d)

    For e=0.001, d=0.01: w=2718, d=5 → ~54KB total!
    """

    def __init__(self, width: int = 2718, depth: int = 5):
        self.width = width
        self.depth = depth
        self.table = [[0] * width for _ in range(depth)]

    def _hash(self, item: str, i: int) -> int:
        """Generate hash for row i."""
        h = hashlib.md5(f"{i}:{item}".encode()).hexdigest()
        return int(h, 16) % self.width

    def add(self, item: str, count: int = 1):
        """Increment count for an item."""
        for i in range(self.depth):
            j = self._hash(item, i)
            self.table[i][j] += count

    def estimate(self, item: str) -> int:
        """
        Estimate count (never underestimates, may overestimate).
        Returns minimum across all hash rows.
        """
        return min(
            self.table[i][self._hash(item, i)]
            for i in range(self.depth)
        )


class TrendingService:
    """
    Tracks trending topics using time-windowed Count-Min Sketches.

    Architecture:
```

PYTHON

```python
    - 1-minute buckets (Count-Min Sketch per minute)
    - Aggregate last 60 minutes for "trending now"
    - Keep a min-heap of top-K hashtags
    """

    def __init__(self, window_minutes: int = 60, top_k: int = 50):
        self.window_minutes = window_minutes
        self.top_k = top_k
        self.sketches: dict[int, CountMinSketch] = {}
        self.candidates: set = set()

    def _current_minute(self) -> int:
        return int(time.time() // 60)

    def record_hashtags(self, hashtags: list[str]):
        """Called for every tweet containing hashtags."""
        minute = self._current_minute()

        if minute not in self.sketches:
            self.sketches[minute] = CountMinSketch()
            self._cleanup_old_buckets(minute)

        for tag in hashtags:
            tag_lower = tag.lower()
            self.sketches[minute].add(tag_lower)
            self.candidates.add(tag_lower)

    def get_trending(self, region: str = "global") -> list[dict]:
        """Get top trending hashtags. Aggregate counts across the time window."""
        current = self._current_minute()
        start = current - self.window_minutes

        tag_counts = []
        for tag in self.candidates:
            total_count = 0
            for minute in range(start, current + 1):
                if minute in self.sketches:
                    total_count += self.sketches[minute].estimate(tag)

            if total_count > 10:  # minimum threshold
                tag_counts.append((tag, total_count))

        tag_counts.sort(key=lambda x: x[1], reverse=True)
        top = tag_counts[:self.top_k]

        return [
            {
                "rank": i + 1,
                "hashtag": f"#{tag}",
                "tweet_count": count,
                "category": "trending"
            }
            for i, (tag, count) in enumerate(top)
        ]

    def _cleanup_old_buckets(self, current_minute: int):
        """Remove sketches older than the window."""
```

```python
        cutoff = current_minute - self.window_minutes - 5
        old_keys = [k for k in self.sketches if k < cutoff]
        for k in old_keys:
            del self.sketches[k]
```

Search Service

```python
"""
Twitter Search: Real-time search across billions of tweets.
Uses Elasticsearch with custom analyzers.
"""

from elasticsearch import Elasticsearch

es = Elasticsearch(["http://es-cluster:9200"])

INDEX_SETTINGS = {
    "settings": {
        "number_of_shards": 10,
        "number_of_replicas": 2,
        "analysis": {
            "analyzer": {
                "tweet_analyzer": {
                    "type": "custom",
                    "tokenizer": "standard",
                    "filter": ["lowercase", "stop", "snowball"]
                }
            }
        }
    },
    "mappings": {
        "properties": {
            "content": {"type": "text", "analyzer": "tweet_analyzer"},
            "user_id": {"type": "long"},
            "username": {"type": "keyword"},
            "hashtags": {"type": "keyword"},
            "mentions": {"type": "keyword"},
            "created_at": {"type": "date"},
            "like_count": {"type": "integer"},
            "retweet_count": {"type": "integer"},
        }
    }
}


class TweetSearchService:

    async def search(self, query: str, filters: dict = None,
                     page: int = 0, size: int = 20) -> dict:
        """
        Search tweets with optional filters.
        Supports: full text, hashtag, user, date range, sort options.
        """
```

PYTHON

```python
        must_clauses = [
            {
                "multi_match": {
                    "query": query,
                    "fields": ["content^2", "hashtags^3", "username"],
                    "type": "best_fields",
                    "fuzziness": "AUTO"
                }
            }
        ]

        filter_clauses = []

        if filters:
            if "from_user" in filters:
                filter_clauses.append(
                    {"term": {"username": filters["from_user"]}}
                )
            if "since" in filters:
                filter_clauses.append(
                    {"range": {"created_at": {"gte": filters["since"]}}}
                )
            if "min_likes" in filters:
                filter_clauses.append(
                    {"range": {"like_count": {"gte": filters["min_likes"]}}}
                )

        sort_by = filters.get("sort", "relevance") if filters else "relevance"

        sort_config = {
            "relevance": [{"_score": "desc"}, {"created_at": "desc"}],
            "recent": [{"created_at": "desc"}],
            "popular": [{"retweet_count": "desc"}, {"like_count": "desc"}]
        }.get(sort_by, [{"_score": "desc"}])

        body = {
            "query": {
                "bool": {
                    "must": must_clauses,
                    "filter": filter_clauses
                }
            },
            "sort": sort_config,
            "from": page * size,
            "size": size,
            "highlight": {"fields": {"content": {}}}
        }

        result = es.search(index="tweets", body=body)

        return {
            "total": result["hits"]["total"]["value"],
            "tweets": [
                {
                    "tweet_id": hit["_id"],
                    **hit["_source"],
                    "highlight": hit.get("highlight", {})
```

```python
            }
            for hit in result["hits"]["hits"]
        ]
    }
```

12.4 Architecture Summary

```
SUMMARY

Complete Twitter System — Key Metrics & Choices:
■■■
```

Aspect	YouTube	WhatsApp	Instagram Feed	Twitter Timeline
Core Problem	Video streaming	Real-time messaging	Photo feed generation	Tweet feed generation
Protocol	HTTP/HLS	WebSocket	HTTP	HTTP (+ SSE)
Read:Write	100:1	1:1	100:1	1000:1
Key Store	S3+CDN	Cassandra	MySQL+Redis	MySQL+Redis
Feed Strategy	Recommend engine	N/A	Hybrid fan-out	Hybrid fan-out
Special Feature	Transcoding pipeline	E2E Encryption Signal Protocol	Stories (TTL) Ranking ML	Trending Count-Min
Hardest Challenge	Storage & bandwidth	Connection management	Celebrity fan-out	Celebrity fan-out

SYS 0 #00

Systems Comparison: Final Summary

Aspect	YouTube	WhatsApp	Instagram Feed	Twitter Timeline
Core Problem	Video streaming	Real-time messaging	Photo feed generation	Tweet feed generation
Protocol	HTTP / HLS	WebSocket	HTTP	HTTP (+ SSE)
Read:Write	100 : 1	1 : 1	100 : 1	1000 : 1
Key Store	S3 + CDN	Cassandra	MySQL+Redis	MySQL+Redis
Feed Strategy	Recommend engine	N/A	Hybrid fan-out	Hybrid fan-out
Special Feature	Transcoding pipeline	E2E Encryption (Signal)	Stories (TTL) Ranking ML	Trending Count-Min
Hardest Challenge	Storage & bandwidth	Connection mgmt (500M WS)	Celebrity fan-out	Celebrity fan-out

Design Pattern	YouTube	WhatsApp	Instagram	Twitter
Write Pattern	Upload → Queue → Process	Direct WebSocket	Post → Fan-out	Tweet → Fan-out
Read Pattern	CDN-first, S3 fallback	WS push / APNs	Redis cache + pull	Redis + celebrity pull
Scale Problem	Storage (500TB/day)	500M WebSockets	Celebrity fan-out	Celebrity fan-out
Consistency	Eventual (views)	Exact-once delivery	Eventual (feed)	Eventual (timeline)
CAP Choice	AP + strong writes	CP (message store)	AP	AP